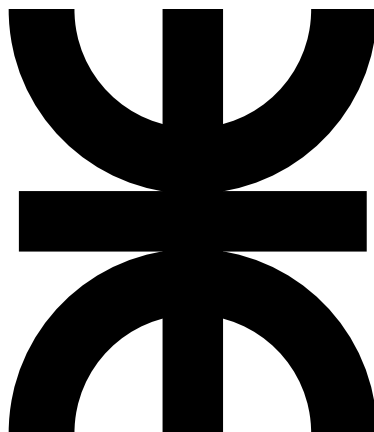


UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

INGENIERÍA ELECTRÓNICA



Técnicas Digitales 1

Guia Practica N°1

Introduccion a Verilog

DOCENTES : Esp.Ing. Marcelo Casasnovas
Ing Florencia Belén Molla JTP

CÁTEDRA : Técnicas Digitales 1

CURSO : 3R4

ALUMNO : Gonza Elías Noel Imanol 87669

CÓRDOBA, ARGENTINA
2026



Índice

1. Ejercicio 1: LED controlado por una llave	3
1.1. Consigna	3
1.2. Entradas	3
1.3. Salidas	3
1.4. Tabla de funcionamiento	3
1.5. Análisis	3
1.6. Código Verilog	3
1.7. Síntesis y RTL	3
1.8. Verificación	4
2. Ejercicio 2: Compuerta AND	5
2.1. Consigna	5
2.2. Entradas	5
2.3. Salida	5
2.4. Tabla de verdad	5
2.5. Análisis	5
2.6. Código Verilog	5
2.7. Síntesis y RTL	6
2.8. Verificación	6
3. Ejercicio 3: Detector de mayoría	7
3.1. Consigna	7
3.2. Entradas	7
3.3. Salida	7
3.4. Tabla de verdad	7
3.5. Análisis	7
3.6. Código Verilog	8
3.7. Síntesis y RTL	8
3.8. Verificación	9
4. Ejercicio 4: Sistema de alarma	10
4.1. Consigna	10
4.2. Entradas	10
4.3. Salida	10
4.4. Tabla de verdad	10
4.5. Análisis	10
4.6. Código Verilog	11
4.7. Síntesis y RTL	11
4.8. Verificación	11
5. Ejercicio 5: Control de acceso	12
5.1. Consigna	12
5.2. Entradas	12
5.3. Salida	12
5.4. Tabla de verdad	12
5.5. Análisis	12



5.6.	Código Verilog	13
5.7.	Síntesis y RTL	13
5.8.	Verificación	13
6.	Ejercicio 6: Control de acceso con supervisor	14
6.1.	Consigna	14
6.2.	Entradas	14
6.3.	Salida	14
6.4.	Tabla de verdad	14
6.5.	Análisis	14
6.6.	Código Verilog	15
6.7.	Síntesis y RTL	15
6.8.	Verificación	15
7.	Ejercicio 7: Multiplexor 2:1	16
7.1.	Consigna	16
7.2.	Entradas	16
7.3.	Salida	16
7.4.	Funcionamiento	16
7.5.	Tabla de verdad completa	16
7.6.	Análisis	17
7.7.	Código Verilog	17
7.8.	Síntesis y RTL	18
7.9.	Verificación	18
8.	Ejercicio 8: Multiplexor 4:1	19
8.1.	Consigna	19
8.2.	Entradas	19
8.3.	Salida	19
8.4.	Funcionamiento	19
8.5.	Tabla de verdad	19
8.6.	Análisis	19
8.7.	Código Verilog	20
8.8.	Síntesis y RTL	21
8.9.	Verificación	22



1. Ejercicio 1: LED controlado por una llave

1.1. Consigna

Se desea diseñar un circuito donde una llave controle directamente un LED.

1.2. Entradas

- A

1.3. Salidas

- Y

1.4. Tabla de funcionamiento

A	Y
0	0
1	1

Cuadro 1: Tabla de funcionamiento del LED controlado por una llave.

1.5. Análisis

A partir de la tabla de funcionamiento se observa que la salida Y toma el mismo valor lógico que la entrada A . Por lo tanto, no se requiere ninguna compuerta lógica adicional, ya que la salida copia directamente el estado de la llave.

La función lógica del circuito es:

$$Y = A$$

1.6. Código Verilog

```
module ejercicio1 (  
    input A,  
    output Y  
);  
  
    assign Y = A;  
  
endmodule
```

1.7. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que la salida Y se encuentra conectada directamente a la entrada A , cumpliendo con el funcionamiento solicitado.

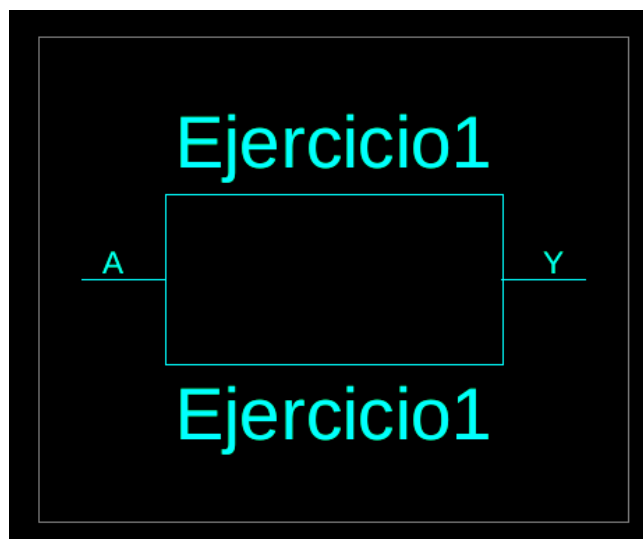


Figura 1: RTL generado para el ejercicio 1.

1.8. Verificación

El funcionamiento esperado es que, cuando la llave conectada a la entrada A se encuentre en estado lógico bajo, el LED permanezca apagado. Cuando la llave se encuentre en estado lógico alto, el LED conectado a la salida Y se encienda.



2. Ejercicio 2: Compuerta AND

2.1. Consigna

Se desea diseñar una compuerta AND de dos entradas.

2.2. Entradas

- A
- B

2.3. Salida

- Y

2.4. Tabla de verdad

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Cuadro 2: Tabla de verdad de la compuerta AND.

2.5. Análisis

A partir de la tabla de verdad se observa que la salida Y toma el valor lógico alto únicamente cuando ambas entradas, A y B , se encuentran en estado lógico alto.

Por lo tanto, la función lógica del circuito es:

$$Y = A \cdot B$$

Esta expresión corresponde a una compuerta AND de dos entradas.

2.6. Código Verilog

```
module ejercicio2 (  
  input A,  
  input B,  
  output Y  
);  
  
  assign Y = A & B;
```

```
endmodule
```

2.7. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que las entradas A y B ingresan a una compuerta AND, cuya salida corresponde a Y .

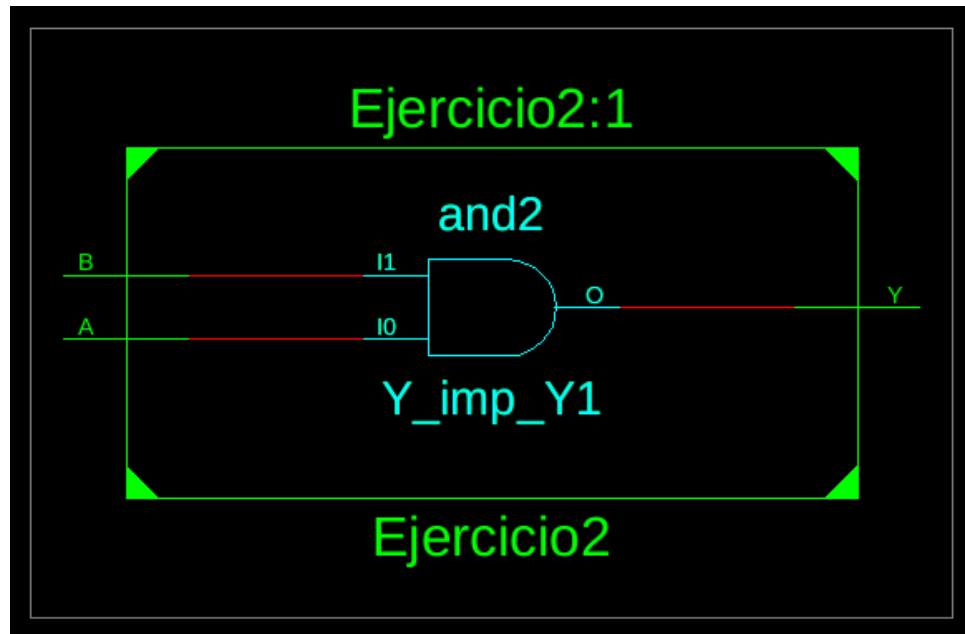


Figura 2: RTL generado para el ejercicio 2.

2.8. Verificación

El funcionamiento esperado es que el LED conectado a la salida Y permanezca apagado cuando alguna de las entradas sea 0. El LED se encenderá únicamente cuando las dos llaves conectadas a A y B estén en estado lógico 1.



3. Ejercicio 3: Detector de mayoría

3.1. Consigna

Se desea diseñar un circuito cuya salida sea igual a 1 cuando al menos dos de sus entradas sean iguales a 1.

3.2. Entradas

- A
- B
- C

3.3. Salida

- Y

3.4. Tabla de verdad

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Cuadro 3: Tabla de verdad del detector de mayoría.

3.5. Análisis

A partir de la tabla de verdad se observa que la salida Y toma el valor lógico alto cuando al menos dos entradas se encuentran en estado lógico 1.

Los casos en los que la salida vale 1 son:

$$ABC = 011, 101, 110, 111$$

Por lo tanto, la función lógica puede expresarse como:

$$Y = A \cdot B + A \cdot C + B \cdot C$$

Esta expresión indica que la salida será 1 si se cumple cualquiera de las siguientes condiciones:

- A y B valen 1.
- A y C valen 1.
- B y C valen 1.

3.6. Código Verilog

```
module ejercicio3 (
  input A,
  input B,
  input C,
  output Y
);

assign Y = (A & B) | (A & C) | (B & C);

endmodule
```

3.7. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que la salida Y se obtiene a partir de la combinación lógica de las entradas A , B y C .

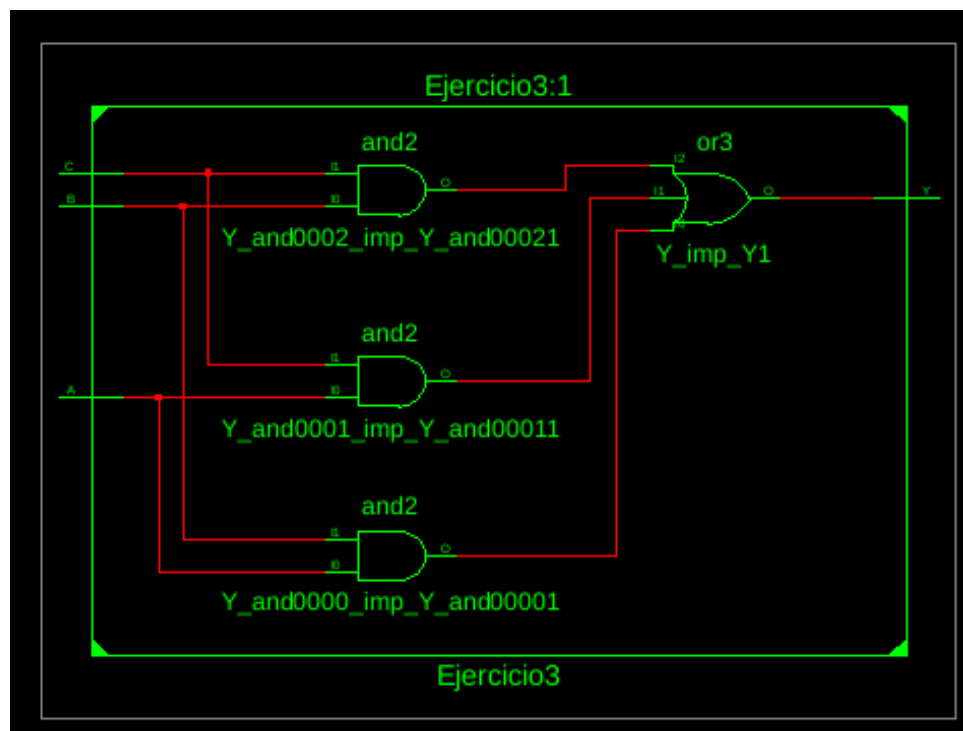


Figura 3: RTL generado para el ejercicio 3.



3.8. Verificación

El funcionamiento esperado es que la salida Y permanezca en estado lógico bajo cuando ninguna entrada, o solamente una entrada, se encuentre en 1. La salida se activa únicamente cuando dos o tres entradas se encuentran en estado lógico alto.



4. Ejercicio 4: Sistema de alarma

4.1. Consigna

Se desea implementar un sistema de alarma para una vivienda. La alarma debe activarse cuando la puerta esté abierta o cuando la ventana esté abierta.

4.2. Entradas

- P : puerta
- V : ventana

4.3. Salida

- A : alarma

4.4. Tabla de verdad

P	V	A
0	0	0
0	1	1
1	0	1
1	1	1

Cuadro 4: Tabla de verdad del sistema de alarma.

4.5. Análisis

A partir de la tabla de verdad se observa que la salida A se activa cuando al menos una de las entradas se encuentra en estado lógico alto.

Si la puerta está abierta, la alarma se activa. Si la ventana está abierta, la alarma también se activa. En caso de que ambas estén cerradas, la alarma permanece desactivada.

Por lo tanto, la función lógica del circuito es:

$$A = P + V$$

Esta expresión corresponde a una compuerta OR de dos entradas.

4.6. Código Verilog

```
module ejercicio4 (
    input P,
    input V,
    output A
);

    assign A = P | V;

endmodule
```

4.7. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que las entradas P y V ingresan a una compuerta OR, cuya salida corresponde a la alarma A .

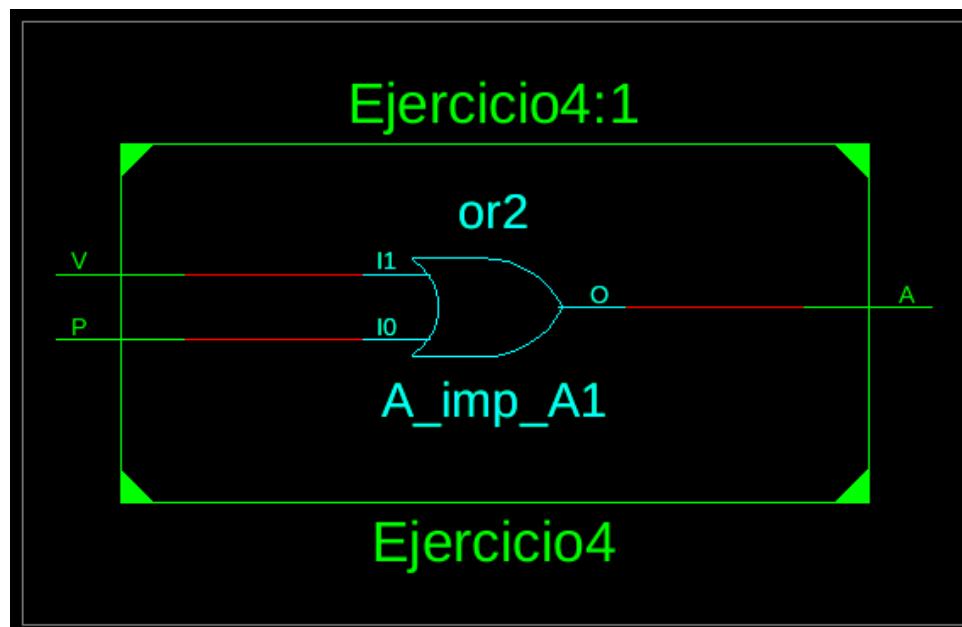


Figura 4: RTL generado para el ejercicio 4.

4.8. Verificación

El funcionamiento esperado es que la alarma permanezca apagada cuando la puerta y la ventana estén cerradas. Si la puerta o la ventana se encuentran abiertas, la salida A pasa a estado lógico alto, activando la alarma.



5. Ejercicio 5: Control de acceso

5.1. Consigna

Se desea implementar un sistema de acceso. El acceso se habilita cuando existe una tarjeta válida y la contraseña es correcta.

5.2. Entradas

- T : tarjeta válida
- C : contraseña correcta

5.3. Salida

- A : acceso habilitado

5.4. Tabla de verdad

T	C	A
0	0	0
0	1	0
1	0	0
1	1	1

Cuadro 5: Tabla de verdad del control de acceso.

5.5. Análisis

A partir de la tabla de verdad se observa que la salida A toma el valor lógico alto únicamente cuando ambas condiciones se cumplen al mismo tiempo: la tarjeta es válida y la contraseña es correcta.

Si una de las dos condiciones no se cumple, el acceso permanece deshabilitado.

Por lo tanto, la función lógica del circuito es:

$$A = T \cdot C$$

Esta expresión corresponde a una compuerta AND de dos entradas.

5.6. Código Verilog

```
module ejercicio5 (
    input T,
    input C,
    output A
);

    assign A = T & C;

endmodule
```

5.7. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que las entradas T y C ingresan a una compuerta AND, cuya salida corresponde a A .

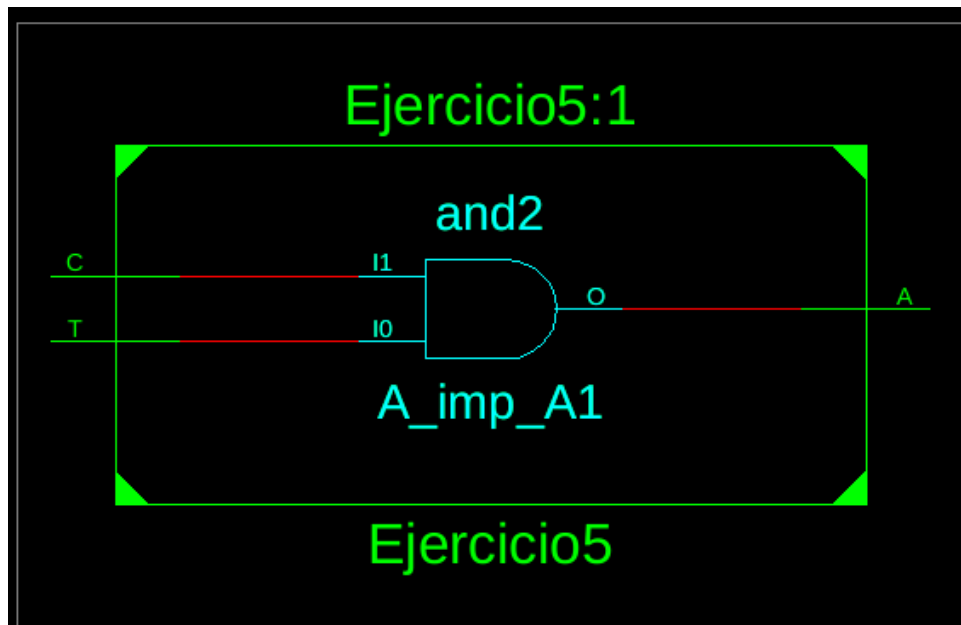


Figura 5: RTL generado para el ejercicio 5.

5.8. Verificación

El funcionamiento esperado es que el acceso permanezca deshabilitado cuando la tarjeta no sea válida o cuando la contraseña sea incorrecta. La salida A se activa únicamente cuando la tarjeta es válida y la contraseña es correcta.



6. Ejercicio 6: Control de acceso con supervisor

6.1. Consigna

Se desea modificar el sistema de acceso del ejercicio anterior considerando que un supervisor puede habilitar el acceso independientemente de la tarjeta y la contraseña.

6.2. Entradas

- T : tarjeta válida
- C : contraseña correcta
- S : supervisor

6.3. Salida

- A : acceso habilitado

6.4. Tabla de verdad

T	C	S	A
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Cuadro 6: Tabla de verdad del control de acceso con supervisor.

6.5. Análisis

A partir de la tabla de verdad se observa que el acceso se habilita cuando se cumplen simultáneamente las condiciones de tarjeta válida y contraseña correcta. Además, el acceso también se habilita si el supervisor se encuentra activo, independientemente del estado de las otras entradas.

Por lo tanto, la función lógica del circuito es:

$$A = T \cdot C + S$$

Esta expresión indica que la salida A será igual a 1 cuando T y C valgan 1, o cuando S valga 1.

6.6. Código Verilog

```
module ejercicio6 (
    input T,
    input C,
    input S,
    output A
);

    assign A = (T & C) | S;

endmodule
```

6.7. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que las entradas T y C ingresan a una compuerta AND, y el resultado se combina mediante una compuerta OR con la entrada S .

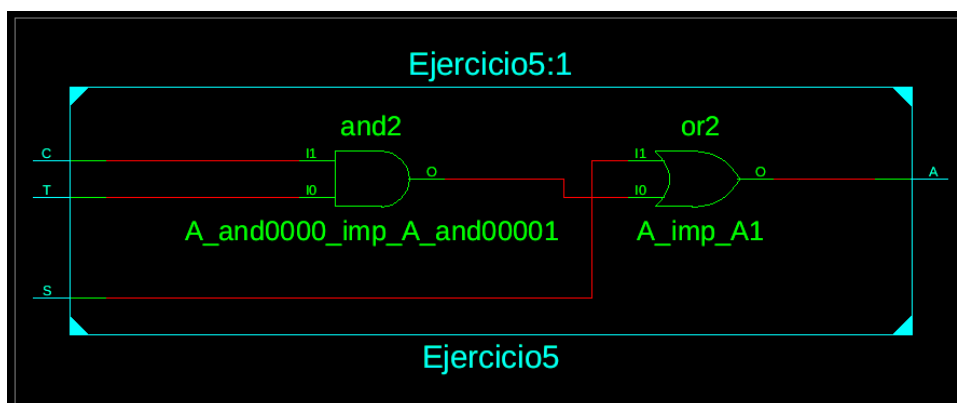


Figura 6: RTL generado para el ejercicio 6.

6.8. Verificación

El funcionamiento esperado es que el acceso permanezca deshabilitado cuando no se cumplan las condiciones de tarjeta válida y contraseña correcta, y cuando el supervisor no esté activo. La salida A se activa si la tarjeta es válida y la contraseña es correcta, o si el supervisor habilita el acceso.



7. Ejercicio 7: Multiplexor 2:1

7.1. Consigna

Se desea diseñar un multiplexor 2 : 1. Este circuito permite seleccionar una de dos entradas de datos y enviarla a una única salida, dependiendo del valor de una señal de selección.

7.2. Entradas

- $I0$: entrada de datos 0
- $I1$: entrada de datos 1
- SEL : señal de selección

7.3. Salida

- Y : salida del multiplexor

7.4. Funcionamiento

El funcionamiento del multiplexor 2 : 1 está determinado por la entrada de selección SEL .

SEL	Y
0	$I0$
1	$I1$

Cuadro 7: Funcionamiento del multiplexor 2 : 1.

7.5. Tabla de verdad completa

SEL	$I0$	$I1$	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Cuadro 8: Tabla de verdad completa del multiplexor 2 : 1.



7.6. Análisis

A partir de la tabla de funcionamiento se observa que la salida Y depende de la señal de selección SEL .

Cuando $SEL = 0$, la salida debe tomar el valor de la entrada $I0$. Para representar esta condición se utiliza el término:

$$\overline{SEL} \cdot I0$$

Cuando $SEL = 1$, la salida debe tomar el valor de la entrada $I1$. Para representar esta condición se utiliza el término:

$$SEL \cdot I1$$

Por lo tanto, la función lógica del multiplexor 2 : 1 queda expresada como:

$$Y = \overline{SEL} \cdot I0 + SEL \cdot I1$$

7.7. Código Verilog

```
module ejercicio7 (  
  input I0,  
  input I1,  
  input SEL,  
  output Y  
);  
  
assign Y = (~SEL & I0) | (SEL & I1);  
  
endmodule
```

También puede implementarse de forma equivalente utilizando el operador condicional ternario:

```
module ejercicio7 (  
  input I0,  
  input I1,  
  input SEL,  
  output Y  
);  
  
assign Y = SEL ? I1 : I0;  
  
endmodule
```

En esta segunda implementación, la instrucción:

```
assign Y = SEL ? I1 : I0;
```

indica que si $SEL = 1$, la salida Y toma el valor de $I1$. En cambio, si $SEL = 0$, la salida Y toma el valor de $I0$. Por lo tanto, ambas formas describen el mismo funcionamiento del multiplexor 2 : 1.

7.8. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que la salida Y se obtiene seleccionando una de las dos entradas, $I0$ o $I1$, según el valor de la señal SEL .

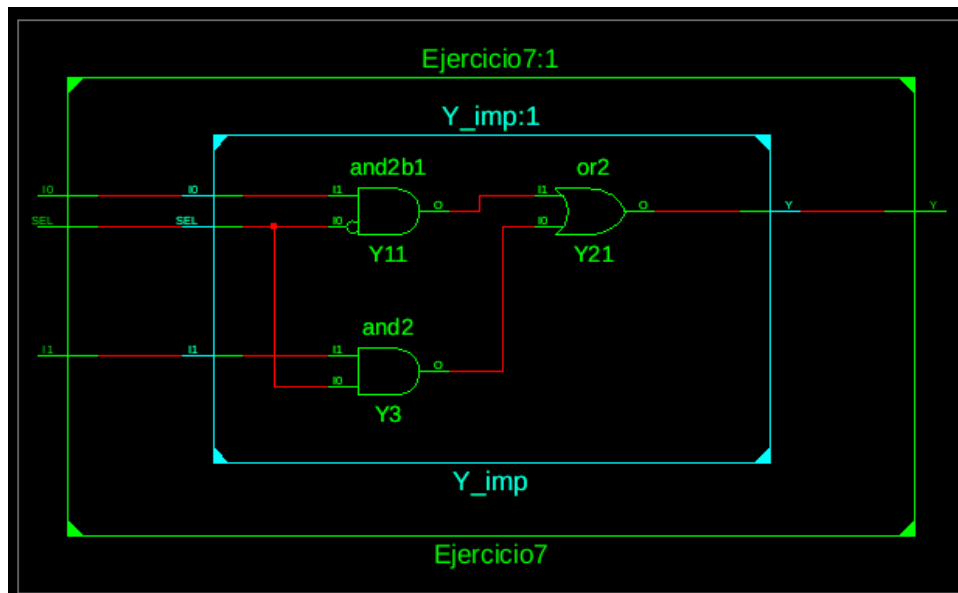


Figura 7: RTL generado para el ejercicio 7.

7.9. Verificación

El funcionamiento esperado es que, cuando la señal SEL se encuentre en estado lógico bajo, la salida Y copie el valor de la entrada $I0$. Cuando SEL se encuentre en estado lógico alto, la salida Y copie el valor de la entrada $I1$.



8. Ejercicio 8: Multiplexor 4:1

8.1. Consigna

Se desea diseñar un multiplexor 4 : 1 utilizando la ecuación lógica correspondiente. Este circuito permite seleccionar una de cuatro entradas de datos y enviarla a una única salida, dependiendo del valor de las señales de selección.

8.2. Entradas

- $I0$: entrada de datos 0
- $I1$: entrada de datos 1
- $I2$: entrada de datos 2
- $I3$: entrada de datos 3
- $S1$: bit más significativo de selección
- $S0$: bit menos significativo de selección

8.3. Salida

- Y : salida del multiplexor

8.4. Funcionamiento

El funcionamiento del multiplexor 4 : 1 depende de las señales de selección $S1$ y $S0$.

$S1$	$S0$	Y
0	0	$I0$
0	1	$I1$
1	0	$I2$
1	1	$I3$

Cuadro 9: Funcionamiento del multiplexor 4 : 1.

8.5. Tabla de verdad

$S1$	$S0$	$I0$	$I1$	$I2$	$I3$	Y
0	0	0	X	X	X	0
0	0	1	X	X	X	1
0	1	X	0	X	X	0
0	1	X	1	X	X	1
1	0	X	X	0	X	0
1	0	X	X	1	X	1
1	1	X	X	X	0	0
1	1	X	X	X	1	1

Cuadro 10: Tabla de verdad resumida del multiplexor 4 : 1.

8.6. Análisis

A partir de la tabla de funcionamiento se observa que la salida Y depende de las señales de selección $S1$ y $S0$.

Cuando $S1 = 0$ y $S0 = 0$, la salida toma el valor de $I0$:

$$\overline{S1} \cdot \overline{S0} \cdot I0$$

Cuando $S1 = 0$ y $S0 = 1$, la salida toma el valor de $I1$:

$$\overline{S1} \cdot S0 \cdot I1$$

Cuando $S1 = 1$ y $S0 = 0$, la salida toma el valor de $I2$:

$$S1 \cdot \overline{S0} \cdot I2$$

Cuando $S1 = 1$ y $S0 = 1$, la salida toma el valor de $I3$:

$$S1 \cdot S0 \cdot I3$$

Por lo tanto, la función lógica del multiplexor 4 : 1 queda expresada como:

$$Y = \overline{S1} \cdot \overline{S0} \cdot I0 + \overline{S1} \cdot S0 \cdot I1 + S1 \cdot \overline{S0} \cdot I2 + S1 \cdot S0 \cdot I3$$

8.7. Código Verilog

El multiplexor 4 : 1 se implementó en Verilog utilizando la ecuación lógica obtenida anteriormente:

```
module ejercicio8 (
input I0,
input I1,
input I2,
```



```
input I3,  
input S1,  
input S0,  
output Y  
);  
  
assign Y = (~S1 & ~S0 & I0) |  
           (~S1 & S0 & I1) |  
           ( S1 & ~S0 & I2) |  
           ( S1 & S0 & I3);  
  
endmodule
```

También puede implementarse de forma equivalente utilizando el operador condicional ternario:

```
module ejercicio8 (  
input I0,  
input I1,  
input I2,  
input I3,  
input S1,  
input S0,  
output Y  
);  
  
assign Y = ({S1,S0} == 2'b00) ? I0 :  
           ({S1,S0} == 2'b01) ? I1 :  
           ({S1,S0} == 2'b10) ? I2 :  
           I3;  
  
endmodule
```

8.8. Síntesis y RTL

Luego de realizar el código en Verilog, se sintetizó el diseño en Xilinx ISE. El RTL generado permite verificar que las señales de selección $S1$ y $S0$ determinan cuál de las cuatro entradas se conecta a la salida Y .

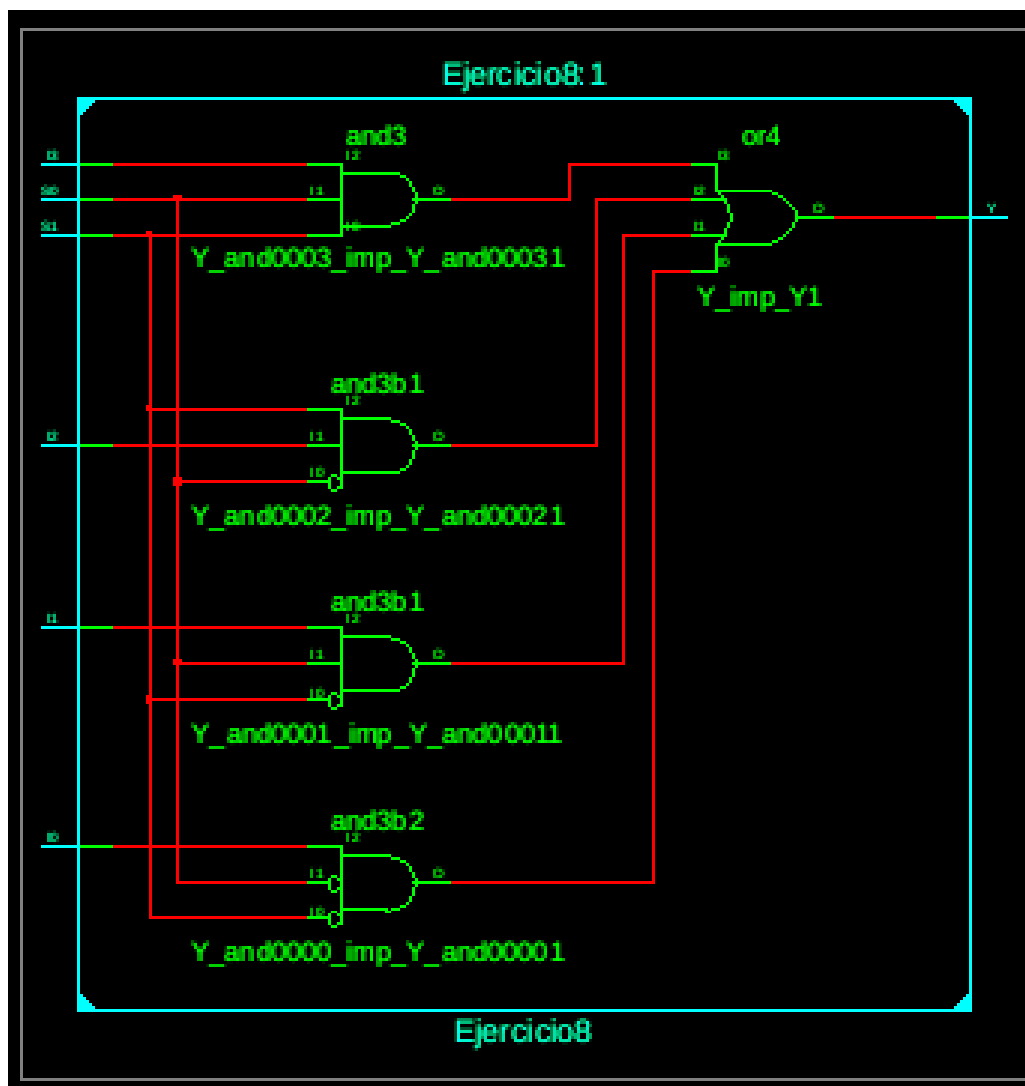


Figura 8: RTL generado para el ejercicio 8.

8.9. Verificación

El funcionamiento esperado es que la salida Y copie el valor de una de las cuatro entradas según el valor de las señales de selección. Si $S1S0 = 00$, la salida toma el valor de $I0$; si $S1S0 = 01$, toma el valor de $I1$; si $S1S0 = 10$, toma el valor de $I2$; y si $S1S0 = 11$, toma el valor de $I3$.